

Los tipos de libro de una biblioteca están representados en el tipo enumerado Libro:

```
public enum Libro {  
    NOVELA, POESIA, AUTOAYUDA, HISTORIA, BIOGRAFIA, CIENCIA;  
}
```

Implemente la clase Biblioteca y los métodos definidos. Debe contener una colección del tipo HashMap que contenga tipo de libro y los días máximos de préstamo permitidos.

```
import java.util.HashMap;  
public class Biblioteca {  
    HashMap <Libro,Integer> mapaDeTiempoRetorno;  
    /**  
    * Inicializa un HashMap para almacenar todos  
    * los libros del enumerado y la cantidad de días  
    * que pueden prestarse.  
    * Debe asignar días por defecto 3  
    */  
    public Biblioteca(){  
        mapaDeTiempoRetorno = new HashMap <Libro, Integer> ();  
        for(Libro libros: Libro.values()){  
            mapaDeTiempoRetorno.put(libros,3);  
        }  
    }  
    /**  
    * Ajusta los días de préstamo de un libro en particular  
    */  
    public void setDias (Libro b, int dias){  
        mapaDeTiempoRetorno.put(b,dias);  
    }  
    /**  
    * Retorna los días de préstamo de un libro en particular  
    */  
    public int getDias(Libro b){  
        return mapaDeTiempoRetorno.get(b);  
    }  
    /**  
    * Retorna el HashMap biblioteca  
    */  
    public HashMap<Libro,Integer> getBiblioteca(){  
        return mapaDeTiempoRetorno;  
    }  
}
```

Una empresa del alquiler de autos posee diferentes alternativas de alquiler para sus clientes. Los tipos de autos están representados en el siguiente enumerado:

```
public enum Auto {  
    ECONOMICO, MEDIANO, GRANDE, LUJO, LIMUSINA;  
}
```

Implemente la clase Catalogo y los métodos definidos. Debe contener una colección del tipo HashMap que contenga tipo de auto y precio de alquiler.

```
import java.util.HashMap;
```

```
public class Catalogo {  
    HashMap<Auto, Integer> catalogo;  
  
    /**  
     * Inicializa un HashMap para almacenar todos los autos  
     * del enumerado y el precio de alquiler. Debe asignar  
     * precio por defecto 1500  
     */  
    public Catalogo(){  
        catalogo = new HashMap<Auto, Integer>();  
        for (Auto a : Auto.values()){  
            catalogo.put(a, 1500);  
        }  
    }  
  
    /**  
     * Ajusta el precio de alquiler de un auto en particular  
     */  
    public void setPrecio (Auto a, int precio){  
        catalogo.put(a, precio);  
    }  
}
```

Un restaurante sirve diferentes bebidas definidas en el tipo enumerado Bebida:

```
public enum Bebida {  
    JUGO, CAFE, CERVEZA, TRAGO, GASEOSA;  
}
```

Implemente la clase Carta y los métodos definidos. Debe contener una colección del tipo HashMap que contenga tipo de bebida y precio.

```
import java.util.HashMap;
```

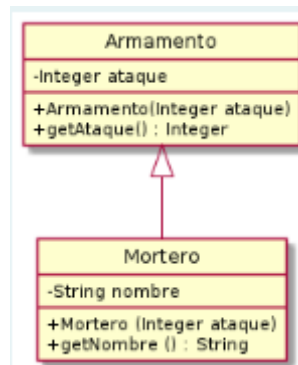
```
public class Carta {  
    HashMap <Bebida,Integer> carta;  
    /**  
     * Inicializa un HashMap para almacenar todas las bebidas del  
     * enumerado  
     * y sus precios. Debe asignar precio por defecto 30  
     */  
    public Carta (){  
        carta = new HashMap<Bebida,Integer>();  
        for(Bebida m : Bebida.values()){  
            carta.put(m,30);  
        }  
    }  
}
```

```

    }
    /**
    * Ajusta el precio de una bebida en particular
    */
    public void setPrecio (Bebida b, int precio){
        carta.put(b,precio);
    }
    /**
    * Retorna el precio de una bebida en particular
    */
    public int getPrecio(Bebida b){
        return carta.get(b);
    }
    /**
    * Retorna el HashMap Carta
    */
    public HashMap<Bebida,Integer> getCarta(){
        return carta;
    }
}

```

Considere el siguiente diagrama de clases, en donde una Mortero es un tipo especial de Armamento que se distingue del resto por su nombre.



Codifique la clase tal que extienda de la clase Armamento, e implemente el constructor y el getter del campo nombre.

```

public class Mortero extends Armamento {
    private String nombre;

    /**
    * Constructor de la clase Mortero.
    * El nombre es el nombre de la clase.
    */
    public Mortero (Integer ataque) {
        super(ataque);
        nombre = "Mortero";
    }

    public String getNombre() {

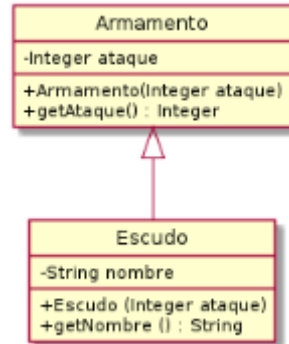
```

```

        return nombre;
    }
}

```

Considere el siguiente diagrama de clases, en donde una Escudo es un tipo especial de Armamento que se distingue del resto por su nombre.



Codifique la clase tal que extienda de la clase Armamento, e implemente el constructor y el getter del campo nombre

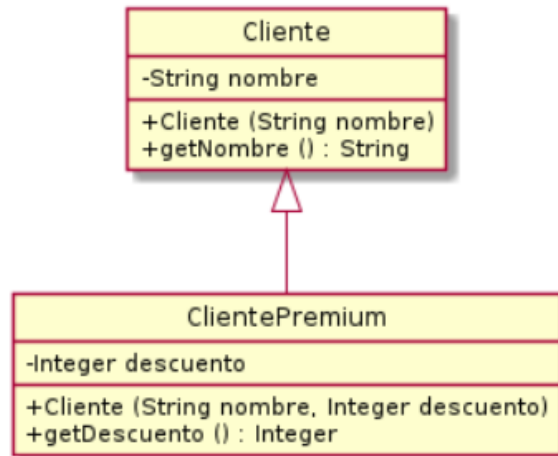
```

//Modifique la definicion de la clase para realizar la herencia
public class Escudo extends Armamento{
    private String nombre;
    //Implemente los siguientes metodos
    /**
    * Constructor de la clase Escudo.
    * El nombre es el nombre de la clase.
    */
    public Escudo (Integer ataque) {
        super(ataque);
        nombre = "Escudo";
    }

    public String getNombre() {
        return nombre;
    }
}

```

Considere el siguiente diagrama de clases, en donde un ClientePremium es un tipo especial de Cliente que posee un descuento en sus compras.



Codifique la clase **ClientePremium** tal que extienda de la clase **Cliente**, e implemente el constructor y el siguiente getter:

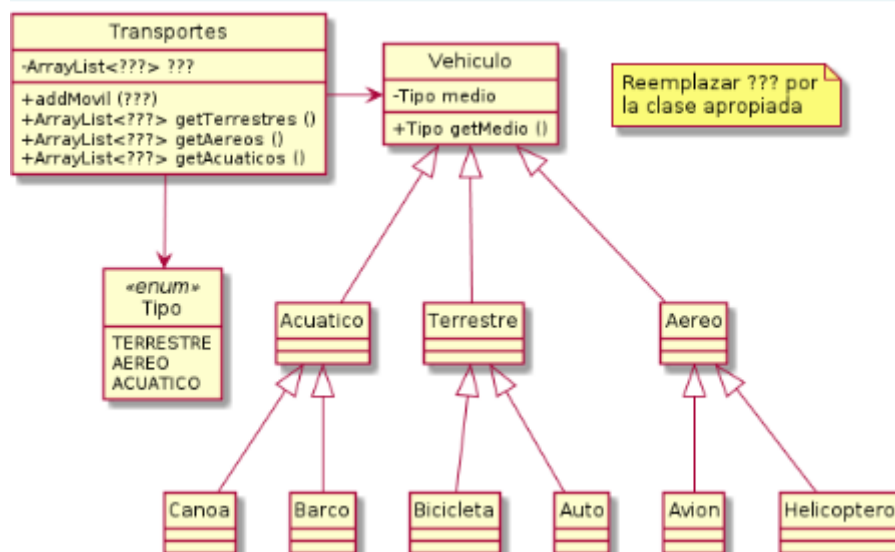
```
public ClientePremium(String nombre, int descuento);
public int getDescuento();
```

```
class ClientePremium extends Cliente {
    //Defina los atributos necesarios de la clase
    private int descuento;

    //Implemente los siguientes metodos
    public ClientePremium(String nombre, int descuento) {
        super(nombre);
        this.descuento = descuento;
    }

    public int getDescuento() {
        return descuento;
    }
}
```

Dado el siguiente diagrama UML



Implemente los métodos indicados y reemplace "???" por lo que corresponda.

```
import java.util.ArrayList;
public class Transportes {
    ArrayList<Vehiculo> moviles;

    public Transportes () {
        moviles = new ArrayList<Vehiculo>();
    }

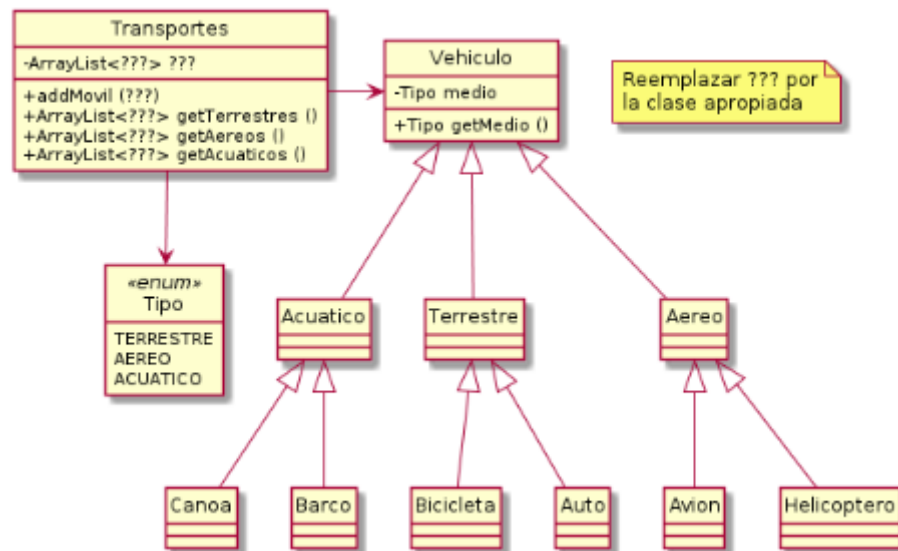
    public void addMovil (Vehiculo v) {
        moviles.add(v);
    }

    public ArrayList<Terrestre> getTerrestres () {
        ArrayList<Terrestre> terrestres = new
ArrayList<Terrestre>();
        for (Vehiculo v : moviles) {
            if (v.getMedio().equals(Tipo.TERRESTRE)) {
                terrestres.add((Terrestre) v);
            }
        }
        return terrestres;
    }

    public ArrayList<Aereo> getAereos () {
        ArrayList<Aereo> aereos = new ArrayList<Aereo>();
        for (Vehiculo v : moviles) {
            if (v.getMedio().equals(Tipo.AEREO)) {
                aereos.add((Aereo) v);
            }
        }
        return aereos;
    }

    public ArrayList<Acuatico> getAcuaticos () {
        ArrayList<Acuatico> acuaticos = new ArrayList<Acuatico>();
        for (Vehiculo v : moviles) {
            if (v.getMedio().equals(Tipo.ACUATICO)) {
                acuaticos.add((Acuatico) v);
            }
        }
        return acuaticos;
    }
}
```

Dado el siguiente diagrama UML



Implemente los métodos indicados y reemplace "???" por lo que corresponda.

```

public class Transportes {
    ArrayList<Vehiculo> moviles;

    public Transportes () {
        moviles = new ArrayList<Vehiculo>();
    }

    public void addMovil (Vehiculo v) {
        moviles.add(v);
    }

    public ArrayList<Acuatico> getAcuaticos () {
        ArrayList<Acuatico> acuaticos = new ArrayList<Acuatico>();
        for(Vehiculo v: moviles){
            if(v.getMedio().equals(Tipo.ACUATICO)){
                acuaticos.add((Acuatico)v);
            }
        }
        return acuaticos;
    }
}

```